

## 说明

CH32H417 是沁恒首款双核通用 MCU，采用青稞 RISC-V 双核架构（V5F + V3F），内置硬件信号量（HSEM）、核间通信（IPC）和物理内存保护（PMP）三大内核级功能。这些功能为双核协同工作、资源安全访问和系统稳定性提供了硬件级支撑，是发挥双核架构优势的基础。本文档整理各功能的核心作用、工作机制和基础使用方法，供二次开发参考。

### 适用范围

| 适用范围   | 系列          |
|--------|-------------|
| 通用 MCU | CH32H417 系列 |

# 目录

|                        |    |
|------------------------|----|
| 说明 .....               | 1  |
| 目录 .....               | 2  |
| 第 1 章 硬件信号量 HSEM ..... | 1  |
| 1.1 功能与作用 .....        | 1  |
| 1.2 原理和使用 .....        | 1  |
| 1.2.1 工作原理 .....       | 1  |
| 1.2.2 使用技巧 .....       | 1  |
| 1.2.3 注意事项 .....       | 2  |
| 第 2 章 核间通信 IPC .....   | 3  |
| 2.1 功能与作用 .....        | 3  |
| 2.2 原理和使用 .....        | 3  |
| 2.2.1 工作原理 .....       | 3  |
| 2.2.2 使用技巧 .....       | 3  |
| 第 3 章 物理内存保护 PMP ..... | 5  |
| 3.1 功能与作用 .....        | 5  |
| 3.2 原理和使用 .....        | 5  |
| 3.2.1 工作原理 .....       | 5  |
| 3.2.2 使用技巧 .....       | 5  |
| 历史版本 .....             | 10 |
| 声明 .....               | 11 |

# 第1章 硬件信号量 HSEM

## 1.1 功能与作用

HSEM (Hardware Semaphore) 是硬件实现的信号量机制，主要用于解决双核系统中的资源竞争与同步问题。

### 核心作用

资源互斥：保护共享外设、共享内存等临界资源，防止双核同时写入导致数据错乱

启动同步：确保 V3F 与 V5F 按预定时序启动，避免访问未初始化资源

低功耗唤醒：释放信号量可触发中断，将另一内核从 STOP/STANDBY 模式唤醒

### 特点

纯硬件逻辑，无软件自旋开销

支持快速单指令获取 (FastTake)

支持中断模式，释放信号量自动触发对方核中断

多个独立信号量通道，可分配给不同资源使用

## 1.2 原理和使用

### 1.2.1 工作原理

每个信号量有两种状态：空闲 (FREE) 和占用 (TAKEN)。

- 获取 (Take)：信号量空闲则立即占用并返回成功；已占用则返回失败 (FastTake 非阻塞模式) 或等待 (阻塞模式)
- 释放 (Release)：释放信号量回到空闲状态，可配置产生中断通知另一内核

### 1.2.2 使用技巧

#### 1) 双核同步

V3F 先运行，初始化系统后唤醒 V5F，然后进入 WFE 等待；V5F 启动后获取并释放信号量，触发中断唤醒 V3F。

```
/****** V3F 核代码 *****/  
  
int main(void)  
{  
    SystemInit();  
  
    // 使能信号量 0 中断  
    HSEM_ITConfig(HSEM_ID0, ENABLE);  
    // 配置事件唤醒 (即使不开中断服务函数也能唤醒)  
    NVIC->SCTLR |= 1 << 4;  
  
    // 唤醒 V5F 核  
    NVIC_WakeUp_V5F(Core_V5F_StartAddr);  
  
    // 进入低功耗等待 V5F 就绪  
    RCC_HB1PeriphClockCmd(RCC_HB1Periph_PWR, ENABLE);  
    PWR_EnterSTOPMode(PWR_Regulator_ON, PWR_STOPEntry_WFE);  
  
    // V5F 释放信号量后唤醒到此处  
    HSEM_ClearFlag(HSEM_ID0);  
}
```

```

while(1) {
    // 主循环
}
}

/***** V5F 核代码 *****/
int main(void)
{
    SystemInit();

    // 快速获取信号量 0
    HSEM_FastTake(HSEM_ID0);
    // 释放信号量 0, 触发 V3F 中断唤醒
    HSEM_ReleaseOneSem(HSEM_ID0, 0);

    while(1) {
        // 主循环
    }
}

```

## 2) 资源互斥访问

访问共享外设（如 SPI、UART）或共享内存前获取信号量，访问完释放。

```

// 访问共享资源前
while(HSEM_ReadStatus(HSEM_ID1) != FREE); // 轮询等待
HSEM_Take(HSEM_ID1); // 获取信号量

// === 临界区：操作共享资源 ===
// 例如：读写共享缓冲区、配置共享外设

HSEM_Release(HSEM_ID1); // 释放信号量

```

### 1.2.3 注意事项

- HSEM\_FastTake 是非阻塞的，获取失败会立即返回，需检查返回值
- 信号量中断可以只用作唤醒，不写中断服务函数（配合 SCTL\_R 事件唤醒位）
- 建议不同资源分配不同信号量 ID，避免嵌套使用造成死锁

## 第2章 核间通信 IPC

### 2.1 功能与作用

IPC (Inter-Process Communication) 是\*\*专用的核间消息邮箱\*\*，提供 4 个独立通信通道，实现双核之间的指令通知和小数据传输。

**核心作用：**

- **消息通知：**一个核向另一个核发送事件信号，触发中断
- **小数据传输：**4\*32bit 数据，适合传命令、状态、地址指针
- **双向通信：**每个通道支持双向收发，中断独立

**与共享内存的区别：**

- IPC 是带中断通知的轻量消息通道，适合传控制指令
- 共享内存（512KB 共享 SRAM）适合批量数据交换
- 实际项目通常组合使用：IPC 发通知 + 共享内存传数据

### 2.2 原理和使用

#### 2.2.1 工作原理

IPC 采用 Mailbox 邮箱机制：

包含 4 个 32 位数据寄存器（MSG0~MSG3）

发送方写入数据后，置位发送寄存器，触发接收方中断

接收方在中断中读取数据，处理后清除中断标志

支持两种中断模式：接收中断、发送中断

#### 2.2.2 使用技巧

典型通信流程

```
#ifndef Core_V3F
    IPC_DeInit();
    IPC_Config(IPC_CH0,IPC_TxCID1,IPC_RxCID0);
    IPC_CH0_Lock();
    IPC_WriteMSG(IPC_MSG0,0);
    IPC_SetFlagStatus(IPC_CH0,IPC_CH_Sta_Bit0);
    IPC_ClearFlagStatus(IPC_CH0,IPC_CH_Sta_Bit1);
    NVIC_SetPriority(IPC_CH0_IRQn,0<<7);
    NVIC_EnableIRQ(IPC_CH0_IRQn);
    Delay_Ms(100);
    IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit1,ENABLE);
#else
    NVIC_SetPriority(IPC_CH0_IRQn,0<<5); //V3 interrupt config
    NVIC_EnableIRQ(IPC_CH0_IRQn);
#endif

//中断服务函数
#ifdef Core_V3F
    if (IPC_GetITStatus(IPC_CH0,IPC_CH_Sta_Bit0) != RESET)
```

```

{
    uint32_t temp = 0;
    temp = IPC_ReadMSG(IPC_MSG0);
    printf("temp - %08x\r\n",temp);
    temp++;
    cnt++;
    IPC_WriteMSG(IPC_MSG0,temp);
}
IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit0,DISABLE);

if(cnt < 10)
{
    IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit1,ENABLE);
}
else
{
    printf("IPC test end\r\n");
}

#else
if(IPC_GetITStatus(IPC_CH0,IPC_CH_Sta_Bit1) != RESET)
{
    uint32_t temp = 0;
    temp = IPC_ReadMSG(IPC_MSG0);
    printf("temp - %08x\r\n",temp);
    temp++;
    IPC_WriteMSG(IPC_MSG0,temp);
}
IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit1,DISABLE);

IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit0,ENABLE);

#endif

```

### 使用建议

- 通道分工：建议为不同业务分配独立通道（如通道 0 = 控制命令、通道 1 = 数据流通知）
- 大数据传输：IPC 只传“数据就绪”通知和地址，实际数据放共享 SRAM
- 中断优先级：IPC 中断优先级不宜过高，避免影响实时外设
- 校验机制：关键命令可增加校验字，防止干扰导致误触发

## 第3章 物理内存保护 PMP

### 3.1 功能与作用

PMP (Physical Memory Protection) 是 RISC-V 标准的\*\*物理内存保护机制\*\*，可以为内存区域设置读、写、执行权限，防止非法访问。CH32H417 的 V5F 和 V3F 内核各内置 4 路 PMP 通道。

**核心作用：**

- 内存隔离：划分代码区、数据区、外设区，设置不同权限
- 防止误操作：阻止程序意外写入 Flash 或只读数据区
- 增强鲁棒性：指针越界、栈溢出等非法访问触发异常，而非静默跑飞
- 安全防护：保护关键配置寄存器不被非授权代码修改

*需要注意的是在 CH32H417 平台上只有 V5F 支持 PMP*

### 3.2 原理和使用

#### 3.2.1 工作原理

每路 PMP 由地址寄存器 (pmpaddr) 和配置寄存器 (pmpcfg) 组成：

- 地址寄存器：定义保护区域的起始地址和范围
- 配置寄存器：设置权限 (R/W/X)、地址匹配模式、锁定位

权限位定义：

- R (Read)：允许读
- W (Write)：允许写
- X (Execute)：允许取指执行
- L (Lock)：锁定配置，锁定后直至复位不可修改

地址匹配模式：

- 4 字节粒度的自然对齐模式
- 支持 TOR (顶部范围)、NA4 (4 字节区域)、NAPOT (自然对齐幂次区域) 三种模式

#### 3.2.2 使用技巧

使用可以参考官方的例程

```
#include "hardware.h"

#ifdef Core_V3F
#warning Core V3 is not support PMP
#endif

/* Global define */
#define PMP_MODE_TOR 0x00000008
#define PMP_MODE_NA4 0x00000010
#define PMP_MODE_NAPOT 0x00000018

#define PMP_AUTHOR_R 0x00000001
#define PMP_AUTHOR_W 0x00000002
#define PMP_AUTHOR_X 0x00000004

#define PMP_LOCK 0x00000080
#define PMP_MODE PMP_MODE_NAPOT
```

```

/* Global Variable */
volatile uint32_t pmp_grop, pmp_mode, pmp_start_address, pmp_end_address;
volatile uint32_t *pmpaddrbase;

/*****
 * @fn SW_Handler
 *
 * @brief The above function is an interrupt handler that sets the PMP (Physical Memory Protection)
address
 * and configuration based on certain conditions.
 *
 * @return none
 */
__attribute__((interrupt("WCH-Interrupt-fast"))) void SW_Handler()
{
    if ((pmp_mode >> (pmp_grop * 8) & 0x00000018) == PMP_MODE_TOR)
    {
        if (pmp_grop == 0)
        {
            __asm volatile("csrw pmpaddr0 , %0" ::"r"(pmp_end_address));
        }
        else if (pmp_grop == 1)
        {
            __asm volatile("csrw pmpaddr0 , %0" ::"r"(pmp_start_address));
            __asm volatile("csrw pmpaddr1 , %0" ::"r"(pmp_end_address));
        }
        else if (pmp_grop == 2)
        {
            __asm volatile("csrw pmpaddr1 , %0" ::"r"(pmp_start_address));
            __asm volatile("csrw pmpaddr2 , %0" ::"r"(pmp_end_address));
        }
        else if (pmp_grop == 3)
        {
            __asm volatile("csrw pmpaddr2 , %0" ::"r"(pmp_start_address));
            __asm volatile("csrw pmpaddr3 , %0" ::"r"(pmp_end_address));
        }
    }
    else if ((pmp_mode & (PMP_MODE_NA4 << (pmp_grop * 8))) || (pmp_mode & (PMP_MODE_NAPOT << (pmp_grop *
8))))
    {
        if (pmp_grop == 0)
        {
            __asm volatile("csrw pmpaddr0 , %0" ::"r"(pmp_start_address));
        }
        else if (pmp_grop == 1)
        {

```



```

        __asm volatile("csrw pmpaddr1 , %0" ::"r"(pmp_start_address));
    }
    else if (pmp_grop == 2)
    {
        __asm volatile("csrw pmpaddr2 , %0" ::"r"(pmp_start_address));
    }
    else if (pmp_grop == 3)
    {
        __asm volatile("csrw pmpaddr3 , %0" ::"r"(pmp_start_address));
    }
}

__asm volatile("csrw pmpcfg0 , %0" ::"r"(pmp_mode));
}

/*****
* @fn PMP_Set_Range
*
* @brief The function `PMP_Set_Range` sets the range and mode for the Physical Memory Protection (PMP)
* feature.
*
* @param grop The parameter "grop" represents the group number for the PMP (Physical Memory
* Protection) setting. It is used to determine which PMP group the range setting will be
applied to.
*
* md The parameter "md" stands for "mode" and it is used to specify the PMP mode.
*
* sd The parameter "sd" stands for "start address". It represents the starting address of the
* memory range for which the PMP (Physical Memory Protection) settings are being configured.
*
* ed The parameter "ed" in the function PMP_Set_Range represents the end address of the range
* for the PMP (Physical Memory Protection) setting. But when you set the PMP mode as
PMP_MODE_NAPOT,
*
* the parameter en means the size of pmp is 2^(en+2). Please refer to the
manual(QingKeV4_Processor_Manual.PDF)
*
* for details
*
* @return none
*/
void PMP_Set_Range(uint32_t grop, uint32_t md, uint32_t sd, uint32_t ed)
{
#ifdef Core_V5F
#warning "Check the pmp setting carefully"
#endif

    pmp_start_address = sd >> 2;
    if ((md & PMP_MODE_NAPOT) == PMP_MODE_NAPOT)
    {
        uint32_t temp = 0xffffffff;
        for (int i = 0; i < ed; i++)
        {

```

```

        temp <= 1;
    }
    pmp_start_address &= temp;
    temp = 0;
    for (int i = 0; i < ed - 1; i++)
    {
        temp <= 1;
        temp |= 0x00000001;
    }
    pmp_start_address |= temp;
}

else
{
    pmp_end_address = ed >> 2;
}

pmp_mode = md << (8 * grop);
pmp_grop = grop;
NVIC_EnableIRQ(Software_IRQn);
NVIC_SetPendingIRQ(Software_IRQn);
}

volatile uint32_t ProtectSec[0x400] = {1, 2, 3, 4, 5, 6, 7};

uint32_t FinalOprateAddress;
extern void Ecall_M_Mode_Handler(void);
/*****
 * @fn      Hardware
 *
 * @brief   Memory protect test
 *
 * @return  none
 */
void Hardware(void)
{
#ifdef Core_V3F
    printf("Core V3 is not support PMP!!\n");
    while(1);
#endif

    printf("PMP TEST\r\n");

    /* The code snippet is setting up the PMP (Physical Memory Protection) range and mode. */
    #if PMP_MODE == PMP_MODE_TOR
        /*USE PMP_MODE_TOR ProtectSec[0x0] - ProtectSec[0x200] is not writable nor excusable*/
        PMP_Set_Range(1, PMP_MODE_TOR | PMP_AUTHOR_R, (uint32_t)(ProtectSec), (uint32_t)&ProtectSec[0x200]);
    #elif PMP_MODE == PMP_MODE_NA4

```

```

/*USE PMP_MODE_TOR (uint32_t)ProtectSec[0] is not writable nor excusable*/
PMP_Set_Range(0, PMP_MODE_NA4 | PMP_AUTHOR_R, (uint32_t)(ProtectSec), (uint32_t)0);
#elif PMP_MODE == PMP_MODE_NAPOT
/*USE PMP_MODE_TOR ProtectSec[0]-ProtectSec[2^(2+2)] is not writable nor excusable*/
PMP_Set_Range(0, PMP_MODE_NAPOT | PMP_AUTHOR_R, (uint32_t)(ProtectSec), (uint32_t)2);
#endif

/* The code snippet is initializing the memory range `ProtectSec` by setting all its elements to 0.
*/
FinalOprateAddress = (uint32_t)&ProtectSec[sizeof(ProtectSec) / sizeof(*ProtectSec) - 1];
for (uint32_t z = (uint32_t)&ProtectSec[sizeof(ProtectSec) / sizeof(*ProtectSec) - 1]; z >
(uint32_t)ProtectSec;
    z -= 1)
{
    FinalOprateAddress = *(uint8_t *)z;
    *(uint8_t *)z = 0;
}

printf("Operation succeed!!\r\n");
while (1)
{
    // printf("%s",__func__);
    Delay_Ms(1000);
}
}

```

需要注意，PMP 相关的配置寄存器操作权限是 MRW 的，因此需要确保在配置的时候已经正确进入了机器模式。官方的例程为了方便在一个.c 文件内演示因此选择在进入 main 函数之后再触发软件中断再进行配置。但是常规的用法应当是在启动文件内配置，后续应当会很少修改。

历史版本

| 日期       | 版本   | 变更内容 |
|----------|------|------|
| 2026/7/8 | V1.0 | 初版发行 |

## 声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。